Aegis Guardrails - TorQ-ops Chat (Demo Reference)

The AI support agent can investigate the kdb+ feeds, but it cannot mutate data, escape to the OS, exfiltrate, read off-limits tables/columns, or run an unbounded scan. Two enforcement layers decide every query: the **GRAMMAR** (which q *shapes* are even expressible) and the **POLICY** (which *tables / columns / tools* are allowed). Everything not explicitly allowed is refused, fail-closed.

Adversarially verified

An audit of 14 attackers ran **1,200+ hostile queries** through the real gate. Result: the **code-execution surface is fully closed** - mutation, shell, eval, IPC, filesystem, namespace, off-allowlist access and value-smuggling are all *structurally unreachable*. The audit also found a resource-governance gap (reducing queries skipped the date-span cap); that has been **fixed** (the span cap now applies to aggregations too, and relative time-windows are magnitude-capped).

Allowed - the safe menu

- **Tables:** `prices_reuters`, `prices_bloomberg`, `prices_exchange`
- **Columns:** `time`, `sym`, `bid`, `ask`, `bsize`, `asize`, `feedsource`
- **Shapes:** `select` / `by` / `where` / `meta`; aggregations (`count`, `sum`, `avg`, `min`, `max`, `first`, `last`, `dev`, `var`, `med`, `wavg`, `wsum`, `countdistinct`, null-counts `sum null bid`); filters (`= < > <= >= in within like`, recent-window `time > .z.p - 00:05:00`)
- **Every query is auto-bounded:** 1,000,000-row cap + date/partition-first + 5-day span cap. It cannot run unbounded.

NOT allowed - q syntax / commands (blocked by the GRAMMAR; raw q never runs)

Category	Examples	Why refused
Mutation / destroy	`delete`, `update`, `insert`, `upsert`, `set`, `.[t;..;:;..]`, `t::x`	only `select`/`meta` heads exist; no assignment
Shell / OS escape	`system "rm -rf /"`, `\l /etc`, `getenv`, `exit`	non-select heads + `\` `/` are illegal tokens
Dynamic eval	`value "..."`, `get`, `eval`, `parse`, `-6!`, `reval`	runs text as code; rejected as non-select / bad token
IPC / network exfil	`hopen :host:port`, `neg[.z.w]...`, URL targets	`./:` handle syntax rejected; no outbound handle
Filesystem	`read0`, `0:`, `save`, `hsym`, `\l`, `../` traversal	path chars `/ . \ $` are illegal tokens
Namespace / introspect	`.z.w`, `.z.u`, `.Q.*`	only the pure now-reads `.z.p`/`.z.d`/`...` survive
Functions / control	`{x}` lambdas, `f[x]`, `' / \` adverbs, `$[]`, `if`/`while`/`do`	`{ } [ ] @ $ ! ' \` are illegal tokens
Multiple statements	`select ... ; system "..."`	one statement only; top-level `;` rejected
Subquery / nested	`from (select ...)`, functional `?[...]`	`from` must name a single table; `?`/brackets rejected
exec (shape change)	`exec sym from ...`	rejected ("use select") to fix the result shape
Cap evasion	self-supplied `i<N`, `999999 sublist`, oversized date range	bounds are injected by the gate, not the agent

Live verdict: every row returns `BLOCK [lifter]` *with the reason shown.*

NOT allowed - tables / columns (blocked by the POLICY)

Item	Examples	Verdict
Off-allowlist table	`positions`, `trade`, `quote`, `orders`	`table '...' not on the query allowlist`
Sensitive column	`secret`, `pnl`, `account_no`, `client_id`	`column '...' not on the allowlist for this table`
Tool not granted	anything outside `grants.tools`	refused before it runs

Live verdict: every row returns `BLOCK [compiler]`.

How to demo it live (1 second, repeatable)

Open **http://localhost:8820/** -> **"Run the banned-list battery"**. It fires both banned lists straight through the gate and shows, for each, the **WITHOUT-vs-WITH** contrast: the exact q that would RUN unchecked is the q Aegis BLOCKS, with the layer (lifter/compiler) and reason. Deterministic, identical every run. Then the legitimate diagnostics being safely **rewritten** to bounded q.

The two-layer summary (one slide)

- **GRAMMAR (lifter)** - controls which q *shapes* exist. Dangerous commands are not in the subset, so they cannot be written. Extend = code change + adversarial tests.
- **POLICY (compiler /** `aegis_policy.fsp.json`**)** - controls which *tables, columns, tools* are allowed. Extend = edit the JSON (in prod, re-sign it). Widening it only ever expands *bounded, read-only* access; it can never create a dangerous operation.